



Task

PROD-18641

Subtask of Online ordering data for new design

OO use cases

Ask Brain² for a presentation, document or prototype

Status	BACKLOG	Assignees	Empty
Dates	Start → Due	Priority	Empty
Time estimate	Empty	Sprint points	Empty
Track time	Start	Tags	Empty

1. Scope

In scope for this project

Ordering only, end to end: entry → order setup → menu → product → cart → checkout → payment → order outcome. Plus **redemption** (coupons, B\$, gift card) and **earning** (rewards display).

Out of scope — owned by Nomnie Studio

- Marketing content, offers/promo merchandising
- tier display
- Order status tracking after order placement
- Member QR code
- Account management

Use case ID convention: UC-<AREA>-<number> (e.g. UC-MENU-04). Please reference these IDs in Figma frame names and in comments so PM, design, and QA all speak the same language.

Priority tags: [P0] = must be in v1 · [P1] = should be in v1 · [P2] = later release · [W] = wish list, not committed (see §15).

2. Glossary

Term	Meaning
Nomnie Studio	The loyalty/marketing landing surface. Shows offers, loyalty signup, balance, tier, marketing content. Always the starting point. Contains the CTA into ordering.
HQ flow	Brand-level entry (e.g. loyalty.nomnie.com/rolld-au). No store pre-selected — user picks order type, then store.
Store-specific flow	Entry already scoped to one store. Typically dine-in / room service, but can also offer pickup, delivery, room.
OUB	The embedded ordering surface, accessed via a store-scoped URL (e.g. ou.abacus.co/en/store/4393169/). → <i>Needs a full-form definition, see §17.</i>
Magic app	Authenticated context where the user's identity is already known — enables saved cards, real-time rewards, B\$ redemption without a separate login step.
Non-magic app	Unauthenticated context. Requires login and/or guest checkout.
B\$	Loyalty currency redeemable against the bill.
Price trigger	An automatic discount applied when qualifying items are in the cart — no coupon code needed.
Upsell / side-sell	A complementary item suggested after a product is added.
Pay later	Order is placed with no payment taken; the guest pays at the counter.
Order type	Pickup/takeaway, Delivery, Dine in, Room service, Catering.

3. Master flow

 [Online-Ordering-Master-Flow.svg](#)

 [Online-Ordering-Mast...r-Flow \(1\).mermaid](#)

4. Entry points — the three doors

Everything downstream is the same; only the setup differs. Design should treat this as **one ordering product with three entry configurations**, not three products.

Use cases

5. Order setup

Purpose: capture everything needed before a menu can be shown — store, order type, and the fulfilment detail for that type.

6. Menu page

Purpose: browse and select. This is the highest-traffic screen — design effort should be weighted here.

Structure to design

- Header: **store name, order type, selected time** — all visible at a glance, and tappable to change
- Category navigation + sub-categories
- Search
- Filters (dietary, allergen, etc.)
- Hamburger / account menu: login, past orders, saved addresses, saved cards
- Persistent cart summary bar

UC-MENU-13 to **UC-MENU-19** are **not built today**. Design should place a "slot" for them in the layout now so they don't force a redesign later. Note the ordering problem: recommendations, favourites, frequently ordered, and reorder could all want to sit at the top of the menu. **A rail-precedence rule is needed** — see §17.

7. Product detail page (PDP)

Purpose: configure and add a product. This is the most rule-heavy screen in the product — variant logic is where most of the complexity lives.

Always shown

Name · Description · Price (live-updating) · Allergens · Image · Quantity · Variants/modifiers · Add to cart

Variant selection rule types — all must be supported

These rules apply at **both product level and variant level**.

Nested variants

Variants can nest **2, 3, and 4 levels deep** (e.g. Meal → Drink → Size → Ice level). This is the single biggest design risk in the project.

8. Upsell, smart side-sell & social proof

Three distinct mechanics that all appear at the same moment (item added to cart). They must not compete for the same space.

8a. Basic upsell

8b. Smart side-sell / meal upgrade [W]

The difference from a basic upsell: this doesn't just suggest an extra item, it **restructures the cart into a bundle** and shows the saving.

8c. Social proof upselling [W]

Two things to resolve before design commits, both in §17: where the attach-rate numbers come from and how fresh they are, and whether a claim like "100% of customers added this" is defensible for the brand's market. A stated statistic is a factual claim to a customer — if it can't be substantiated per store per period, softer phrasing ("popular pairing") is the safer pattern and design should be briefed accordingly.

9. Price triggers (automatic discounts, no code)

The intent: the guest should understand, **before and after adding**, that qualifying items earn a discount — without hunting for a coupon code.

Design note: badge collision on the menu card is a real risk. There are at least five possible badges (sold out, not for sale, limited special, coming soon, qualifies for offer). Please define badge precedence.

10. Cart

11. Checkout, payment & order outcome

11a. Authentication at checkout

11b. Payment

11c. Order processing & outcome

12. Cross-cutting state library

Design once, reuse everywhere. Please deliver these as components.

Also needed: accessibility pass (contrast, focus order, screen reader labels on variant groups and counters), responsive behaviour for the OUB embed, and a full error-copy deck — there are roughly 20

distinct error messages in this product and they should be written deliberately, not ad hoc.

13. Configuration matrix — what varies

The UI must flex on all of these. Nothing below can be hardcoded.

Design implication: the most demanding version of every screen — dine-in + catering + 4-level variants + 5 checkboxes + 3 active triggers + guest-only — should be designed explicitly, not assumed. Please produce a "maximum config" frame for the menu, PDP, cart, and checkout.

14. Auth × payment permutation matrix

15. Wish list & roadmap — design for the space, don't build

These are things we **want or wish to have**, not v1 commitments. The ask for design is to prove the pattern fits the layout, so adding them later is an increment rather than a redesign.

Permanently out of scope (Nomnie Studio owns it): order status tracking, tier, marketing content, member QR.

Two structural risks the wish list introduces — worth designing against now, cheaply:

Rail congestion at the top of the menu. Recommendations, favourites, frequently ordered, and reorder are four competing rails. Without a precedence rule the menu becomes all rails and no menu.

Persuasion stacking at add-to-cart. A single "add" could fire an upsell, a meal upgrade, a social-proof label, and a price-trigger progress update simultaneously. Design should define **one moment, one message** and a priority order for which mechanic wins.



▼ Fields

Request Type	—
Link to Jira	—

MoSCoW	—
Resolution Status	—
Product Area (Other)	—
Link to PRD	—
Link to TDD	—
Prototype	—
Squad	—
Onboarding Steps	—
Blocker Level	—
What did you expect to...	—
How often	—
Workaround used	—
What happened	—
Screenshot / attachment	—
Issue Type	—
R: Company OKR	—
Progress Updates	- Task was created by @Vivek Dilip Khairnar and set to backlog. - No additional progress or updates have been made.
Technical Lead	—
Release Status	—
Phase	—
Product Lead	—
Summary	—
Country	—
Ordering Stack	—
Internal Release Notes	Release Note: OO use cases  RELEASE DATE: 08/05/2026 Feature/Product: OO use cases
GTM Launch Status	—
Sales Agent	—
GTM Plan	—
Onboarding Agent	—
Support	—
Blog	—
Pricing & Packaging	—

  MRD

Market Requirements Document (MRD)

1. Executive Summary

- No product or feature details provided. Unable to summarize strategic intent or business objectives....

 Item Type	—
 Traffic Light	—
 Feature name	—

 Add subtask

▼ Attachments 2


Online-Orderin... .svg 


Online-Or... .mermaid 

Activity



- Vivek Dilip Khairnar created this task Aug 5 at 5:53 pm
- > Show more

 Vivek Dilip Khairnar Aug 6 at 2:18 pm

@Callum Taylor as discussed, these are the use cases and how OO works



Reply